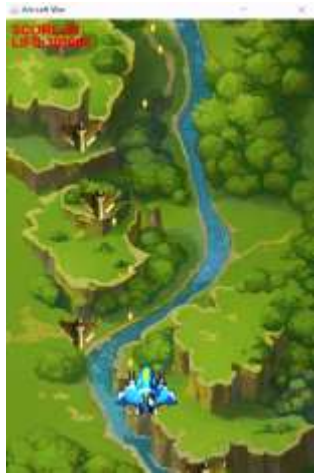
实验三：策略模式

实验与创新实践教育中心 • 计算机与数据技术实验教学部

本学期实验总体安排

初始版本



最终版本

初始版本

游戏主界面
英雄机移动
英雄机子弹直射
碰撞检测
统计得分和生命值

01

UML建模
精英敌机生成、攻击
道具生成、加血道具生效
采用**单例模式**创建英雄机

02

采用**简单工厂模式**
创建道具
采用**工厂方法模式**
创建敌机

03

采用**策略模式**
实现不同飞机的发射弹道
两种火力道具生效

04

采用**数据访问对象模式**
实现得分排行榜
添加**JUnit单元测试**

05

使用**Swing**添加游戏难度选择和排行榜界面
使用**多线程**实现音效控制及火力道具状态恢复

06

采用**观察者模式**
实现炸弹、冰冻道具
采用**模板模式**
实现三种游戏难度

本学期实验总体安排

实验项目	一	二	三	四	五	六
学时数	2	2	2	2	4	4
实验内容	系统分析 单例模式	简单工厂 工厂方法模式	**策略模式**	JUnit 数据访问 对象模式	Swing 多线程	观察者模式 模板模式
时间节点			**中期**			**结项**
提交内容			**UML类图 项目代码**			结项报告 代码、类图 展示视频

实验课程共 16 个学时，6 个实验项目，总成绩为 30分 。

目录

CONTENTS

01 实验目的

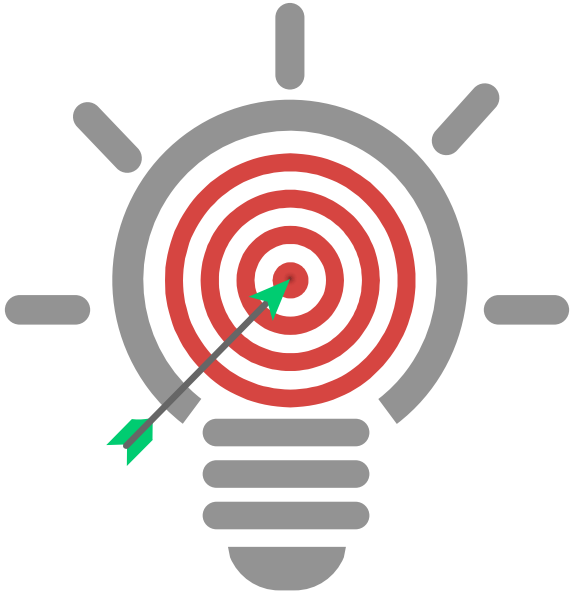
02 实验任务

03 实验步骤

04 中期提交

实验目的

- 深入理解**策略模式**的设计意图及结构
- 结合案例，规范绘制策略模式的**UML类图**
- 掌握**重构**方法，通过**代码实现**并应用策略模式



实验任务

1. 深入理解**策略模式**的设计原理与结构，并结合实例绘制该模式的 UML 结构图；

2. 使用策略模式**重构飞机弹道系统**，实现不同机型的弹道发射逻辑，以及两种火力道具的加成效果；

3. 完成本次迭代开发清单中的全部任务。

课前小测

请思考：

在面向对象编程中，如何实现代码复用？（多选）

A . 封装

B . 继承

C . 组合

答案：B C

继承：在基类的基础上创建新类，新类可直接复用基类的属性和方法；

组合：在新类中创建已有类的对象，通过该对象来调用已有类中的属性和方法。

实验步骤：弹道系统场景分析（1）

弹道系统场景分析

飞机大战游戏中，飞机射击行为具有较强的多样性与动态变化特征，不同机型的弹道各不相同，且英雄机在获得火力道具后会动态切换弹道。

类型		弹道
普通敌机		不发射
精英敌机		单排直射
精锐敌机		双排直射
王牌敌机		散射（同时发射3颗子弹）
Boss敌机		环射（同时发射20颗子弹）
英雄机	无道具	直射
	火力道具	散射（同时发射3颗子弹）
	超级火力道具	环射（同时发射20颗子弹）

实验步骤：弹道系统场景分析（2）

请思考：

1. 目前各种飞机的子弹发射在哪个类实现？

直射

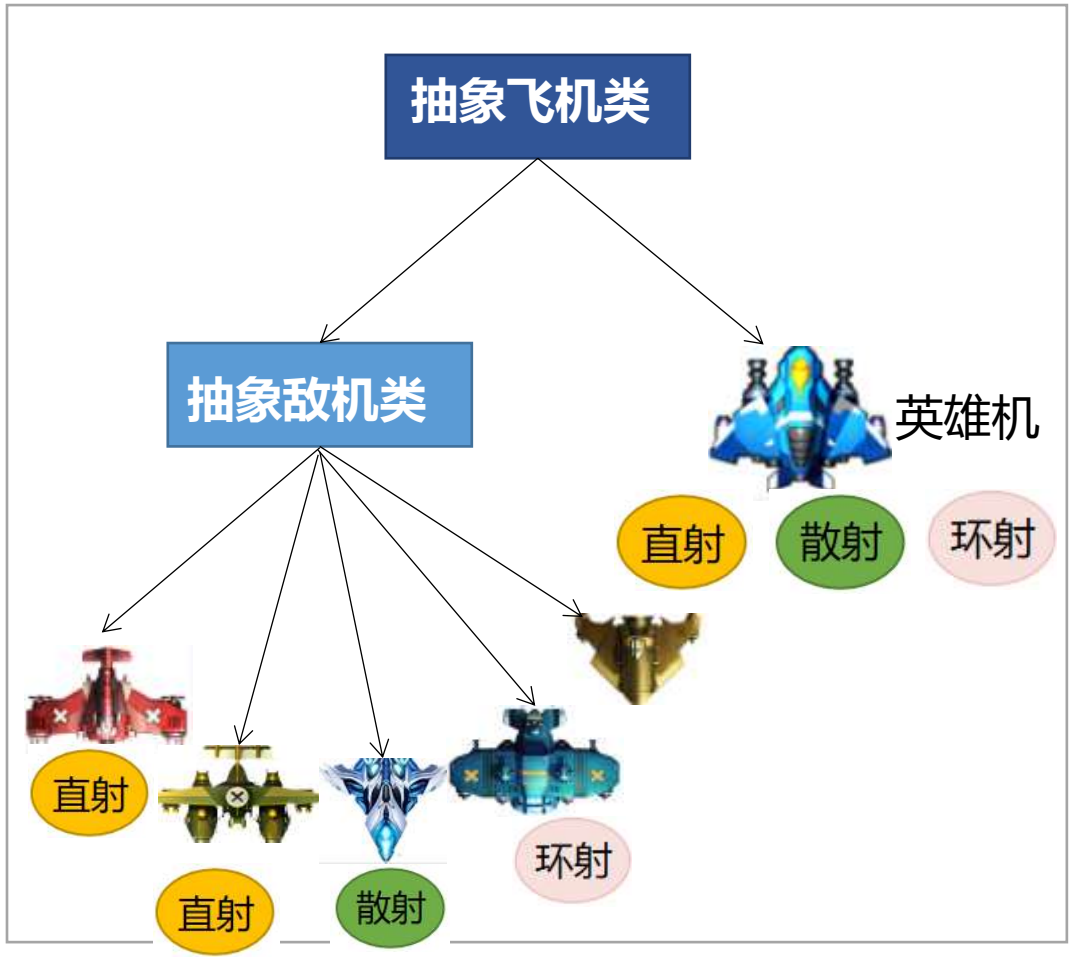
```java
@Override
public List<BaseBullet> shoot() {
    List<BaseBullet> res = new LinkedList<>();
    int x = this.getLocationX();
    int y = this.getLocationY() + direction*2;
    int speedX = 0;
    int speedY = this.getSpeedY() + direction*5;
    BaseBullet bullet;
    for(int i=0; i<shootNum; i++){
        bullet = new HeroBullet( locationX: x + (i*2 - shootNum + 1)*10,
        y, speedX, speedY, power);
        res.add(bullet);
    }
    return res;
}
```

散射

```java
@Override
public List<BaseBullet> shoot() {
    List<BaseBullet> res = new LinkedList<>();
    int x = this.getLocationX();
    int y = this.getLocationY() + direction*2;
    int speedX = this.getSpeedX();
    int speedY = this.getSpeedY() + direction*5;
    BaseBullet bullet;
    for(int i=0; i<shootNum; i++){
        // 多个子弹横向分散，呈扇形
        bullet = new EnemyBullet( locationX: x + (i * 2 - shootNum + 1) * 10, y,
                speedX: speedX + (i * 2 - shootNum + 1), speedY, power);
        res.add(bullet);
    }
    return res;
}
```

实验步骤：弹道系统场景分析（3）

请思考： 2. 如何实现火力道具？如何增加新的机型或弹道？



01 方案一

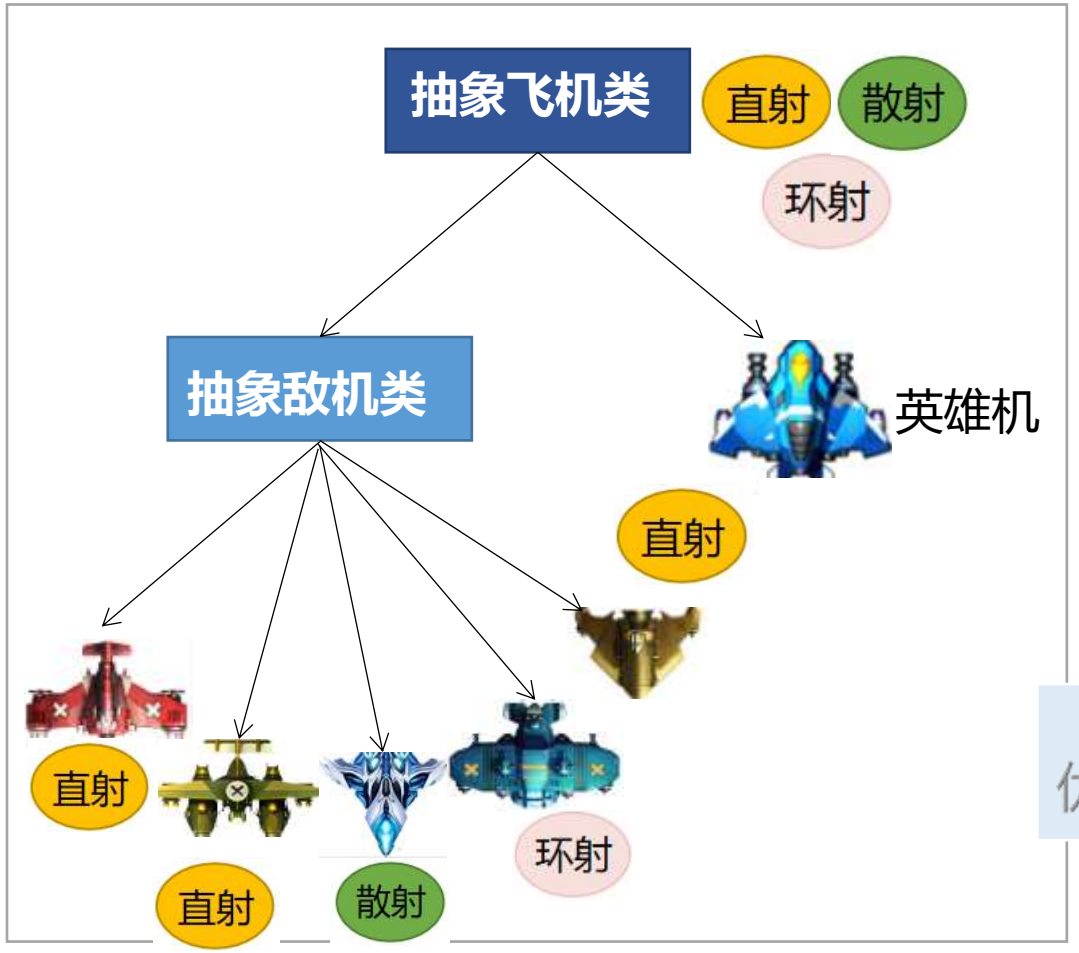
将散射和环射代码复制一份到英雄机类

重复代码多
代码难维护
容易引入bug

违反
开闭原则 X

实验步骤：弹道系统场景分析（3）

请思考： 2. 如何实现火力道具？如何增加新的机型或弹道？



02 方案二

将三种弹道的代码放到父类来继承

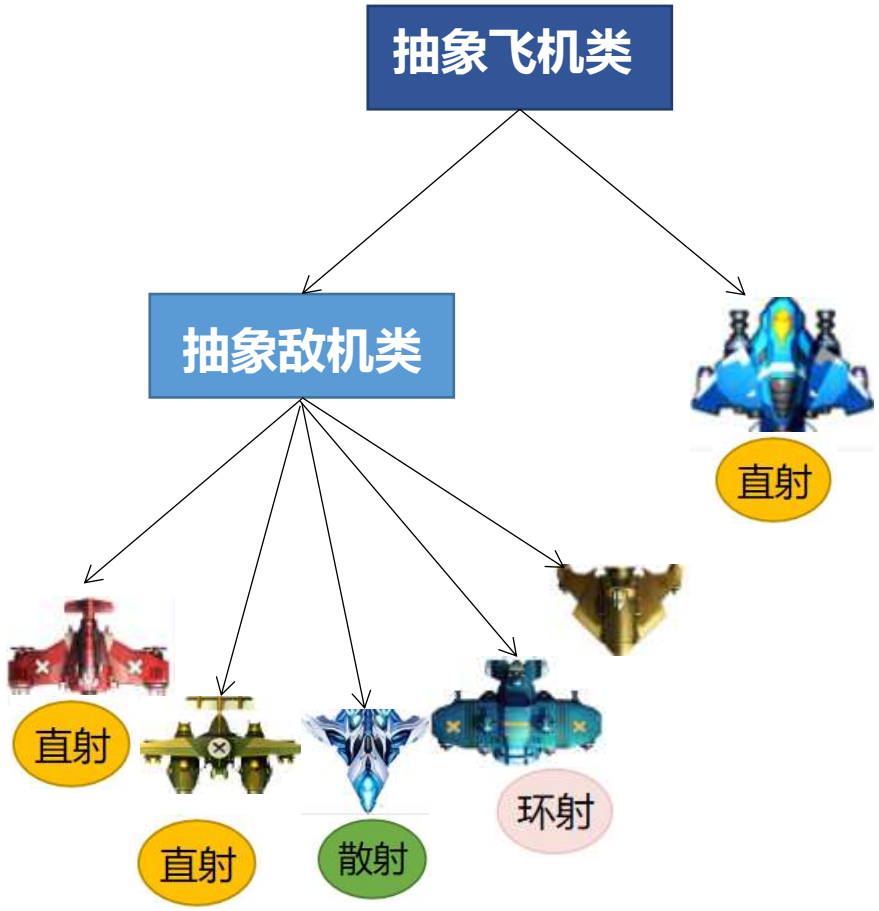
多重选择
逻辑复杂
代码可读性差

违反合成复用原则 X

优先使用**对象组合**，而不是继承来实现代码复用！

实验步骤：弹道系统场景分析（4）

请思考：如何使用对象组合而不是继承来实现代码复用？

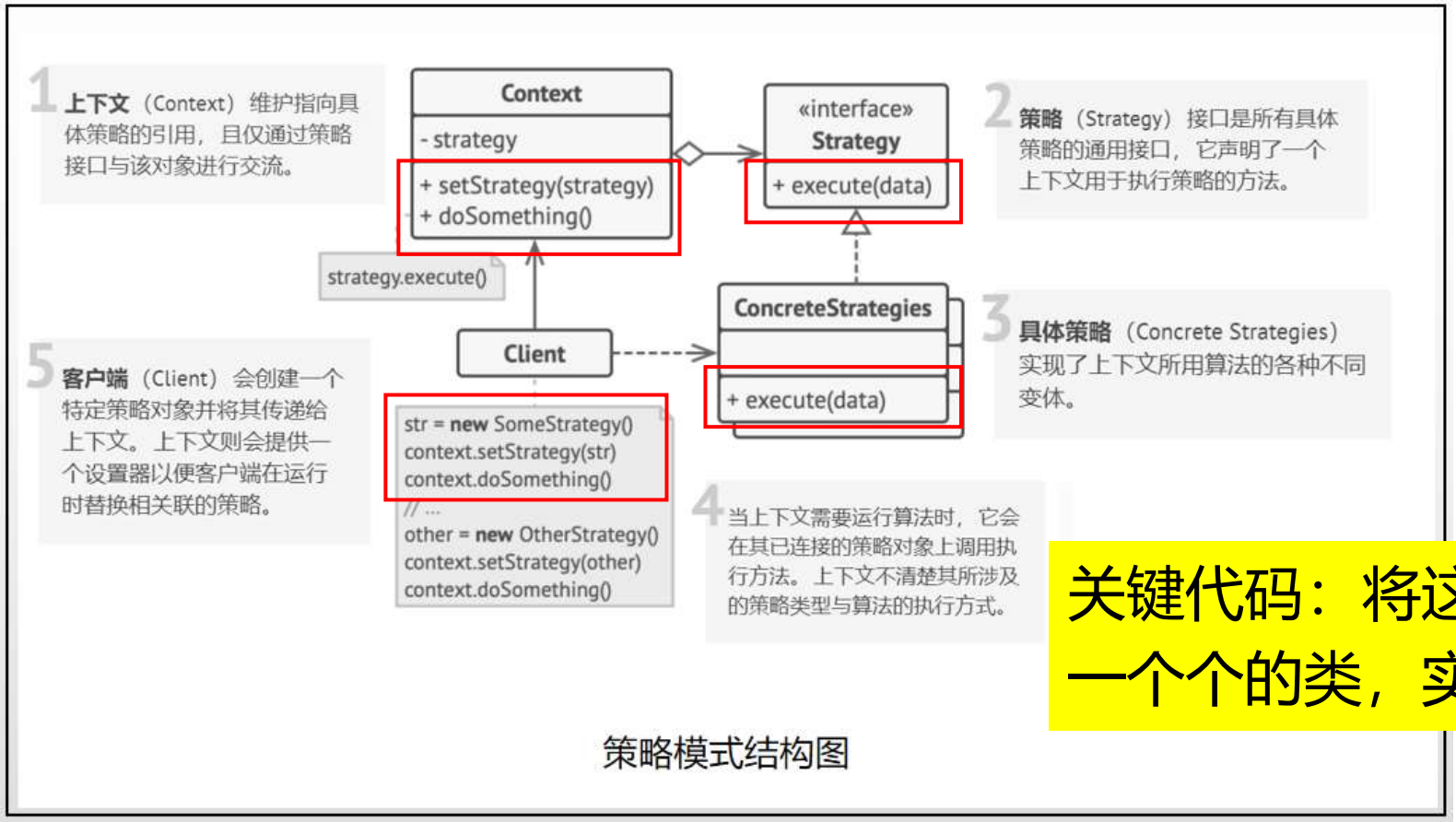


所有设计模式的实现都遵循一条原则：

找出程序中变化的地方，并将变化封装起来！

实验步骤：策略模式结构图

策略模式（Strategy Pattern）是一种行为设计模式， 它能让你定义一系列算法，并将每种算法分别放入独立的类中，以使算法的对象能够相互替换。



策略模式结构图

关键代码：将这些算法封装成一个个的类，实现同一个接口。

实验步骤：计算器举例（策略模式）

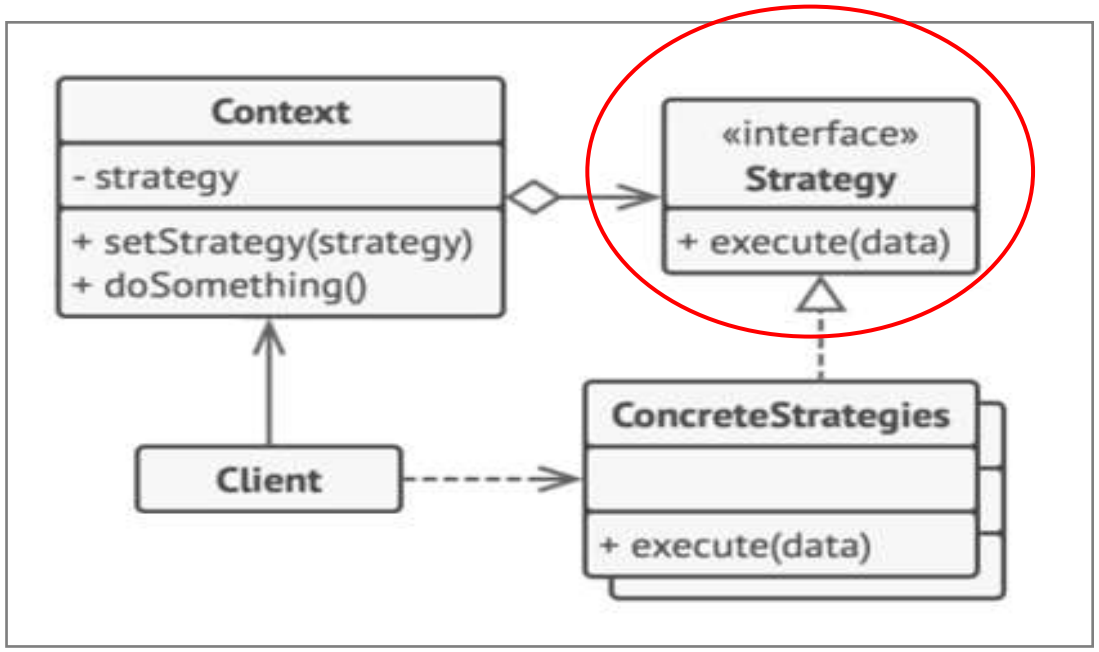
假如我们要实现一个计算器，支持加法、减法、乘法、除法四种运算。我们该如何用策略模式实现呢？

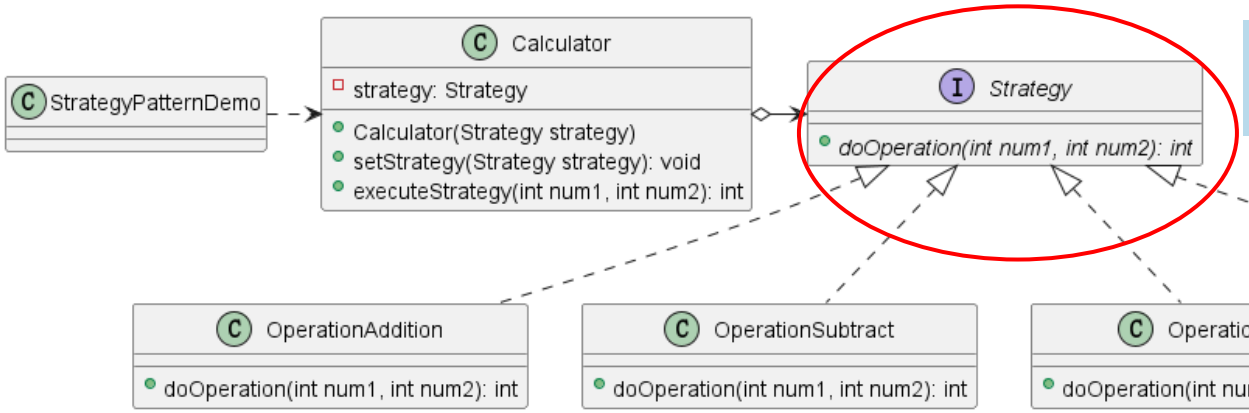
实验步骤：计算器举例（1）

代码实现

① 创建一个Strategy接口，充当抽象策略角色

```
public interface Strategy {
    int doOperation(int num1, int num2);
}
```





UML 类图设计

实验步骤：计算器举例（2）

代 码 实 现

② 创建四个**运算实体类**，充当具体策略角色

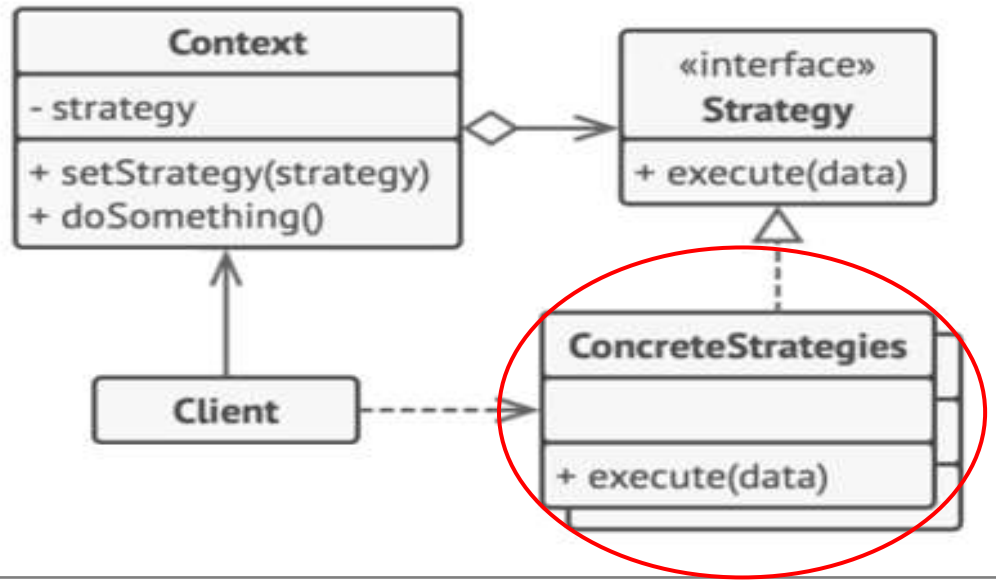
```java
public class OperationAddition implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 + num2;
    }
}
```

加法

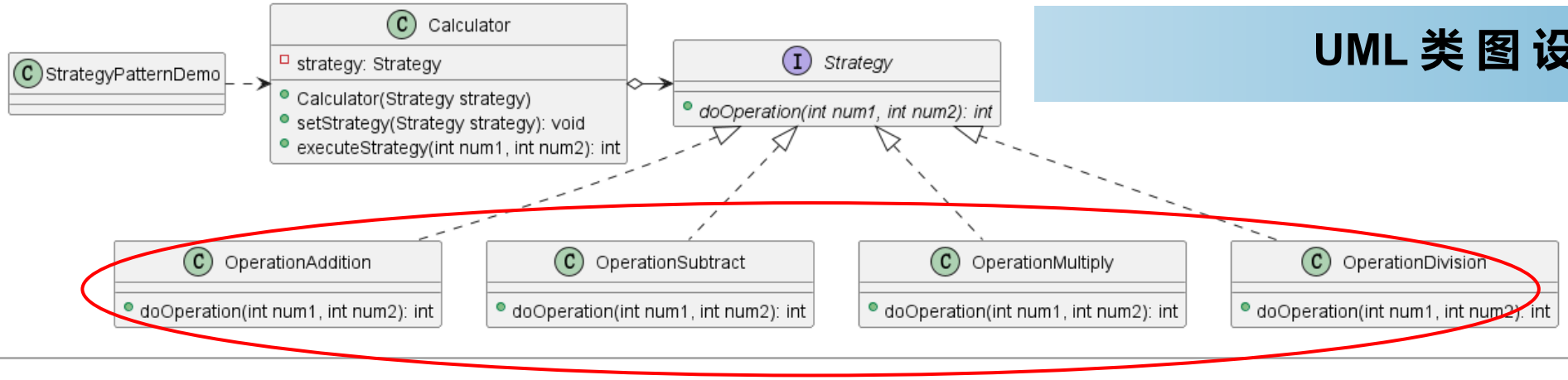
```java
public class OperationSubtract implements Strategy{

    @Override
    public int doOperation(int num1, int num2) {
        return num1 - num2;
    }
}
```

减法

…



UML 类 图 设 计

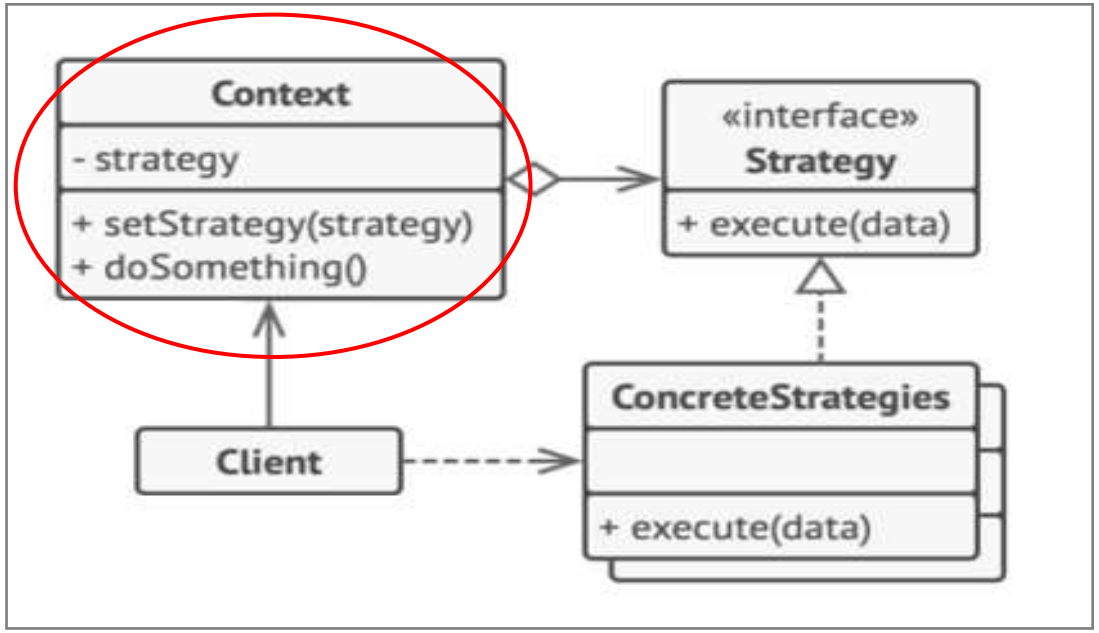


实验步骤：计算器举例（3）

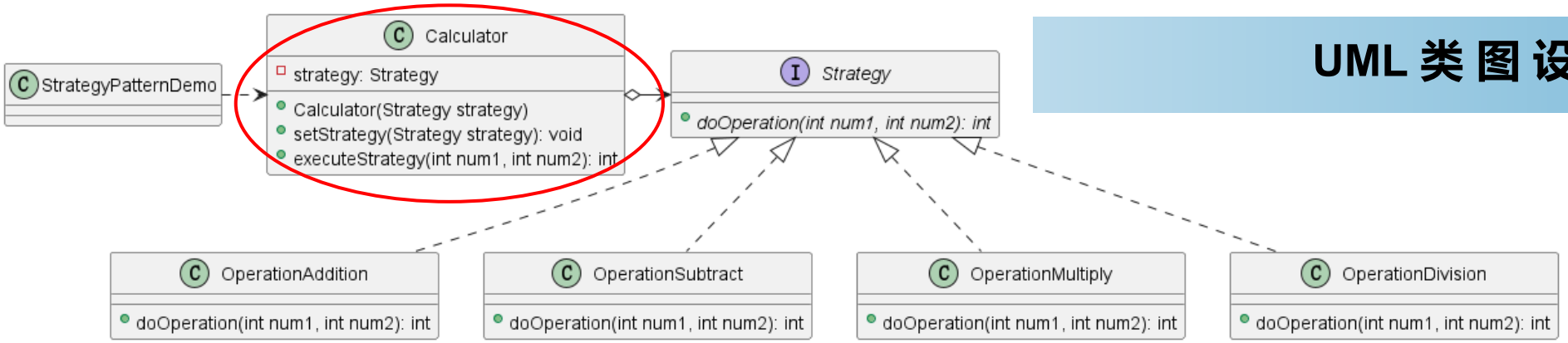
代 码 实 现

③ 创建Calculator 类，充当上下文角色

```java
public class Calculator {
    private Strategy strategy;

    public Calculator(Strategy strategy){
        this.strategy = strategy;
    }
    public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }
    public int executeStrategy(int num1, int num2){
        return strategy.doOperation(num1, num2);
    }
}
```



UML 类 图 设 计

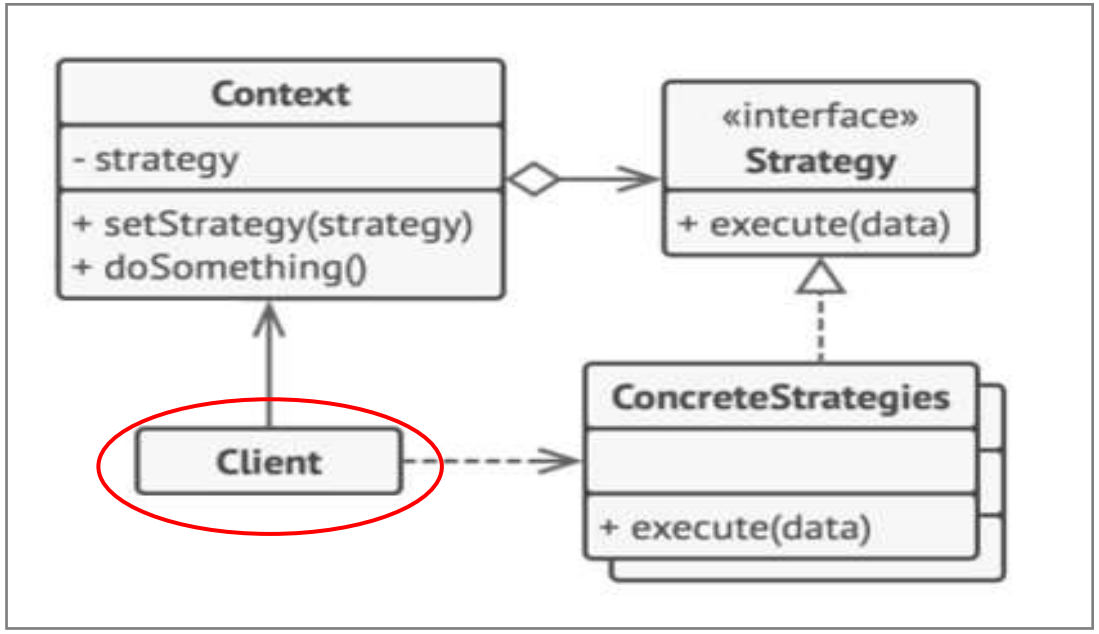


实验步骤：计算器举例（4）

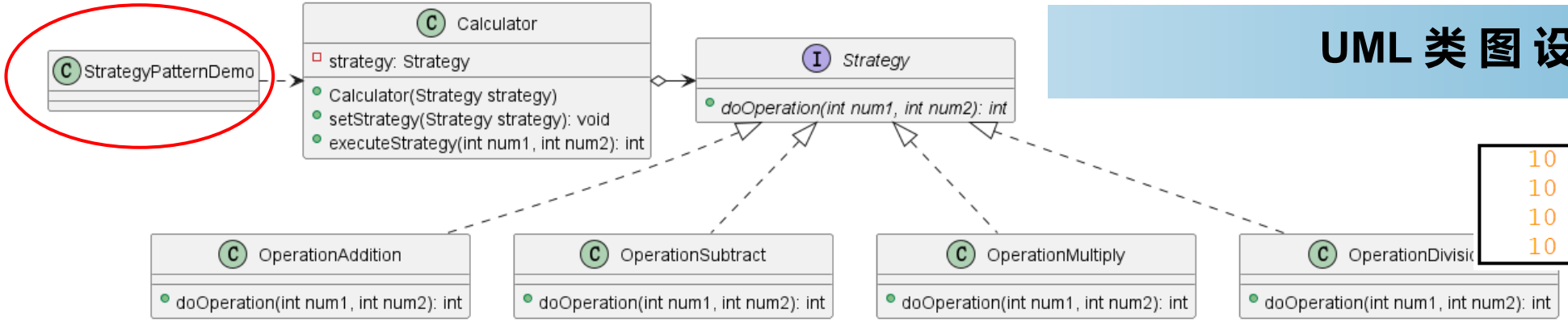
代 码 实 现

④ 客户端使用 Calculator来设置不同的策略

```java
public class StrategyPatternDemo {
    public static void main(String[] args) {

        Calculator calc= new Calculator(new OperationAddition());
        System.out.println("10 + 5 = " + calc.executeStrategy(10, 5));

        calc.setStrategy(new OperationSubtract());
        System.out.println("10 - 5 = " + calc.executeStrategy(10, 5));

        calc.setStrategy(new OperationMultiply());
        System.out.println("10 * 5 = " + calc.executeStrategy(10, 5));
        calc.setStrategy(new OperationDivision());
        System.out.println("10 / 5 = " + calc.executeStrategy(10, 5));
    }
}
```



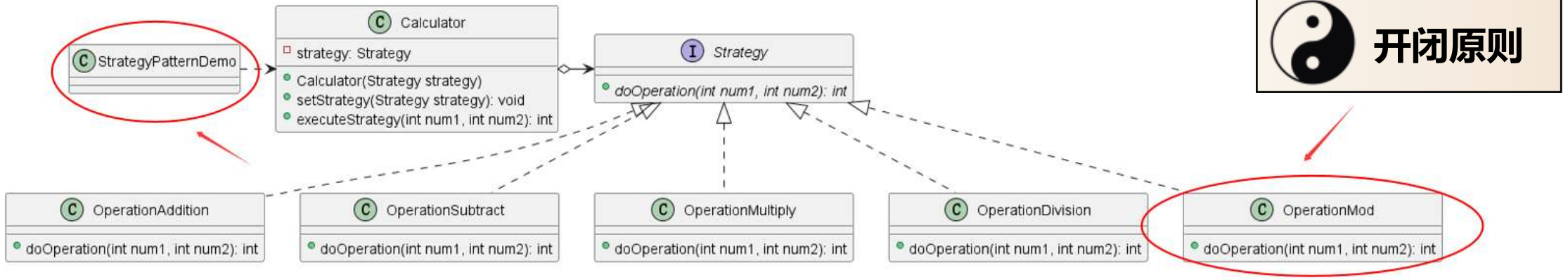
UML 类 图 设 计




```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2
```

实验步骤：计算器举例（5）

请思考：如何添加一个取模运算？




```java
public class StrategyPatternDemo {
    public static void main(String[] args) {
        Calculator calc= new Calculator(new OperationAddition());
        System.out.println("10 + 5 = " + calc.executeStrategy(10, 5));
        calc.setStrategy(new OperationDivision());
        System.out.println("10 / 5 = " + calc.executeStrategy(10, 5));

        //Switch to the modulo operation strategy
        calc.setStrategy(new OperationMod());
        System.out.println("10 % 5 = " + calc.executeStrategy(10, 5));
    }
}
```

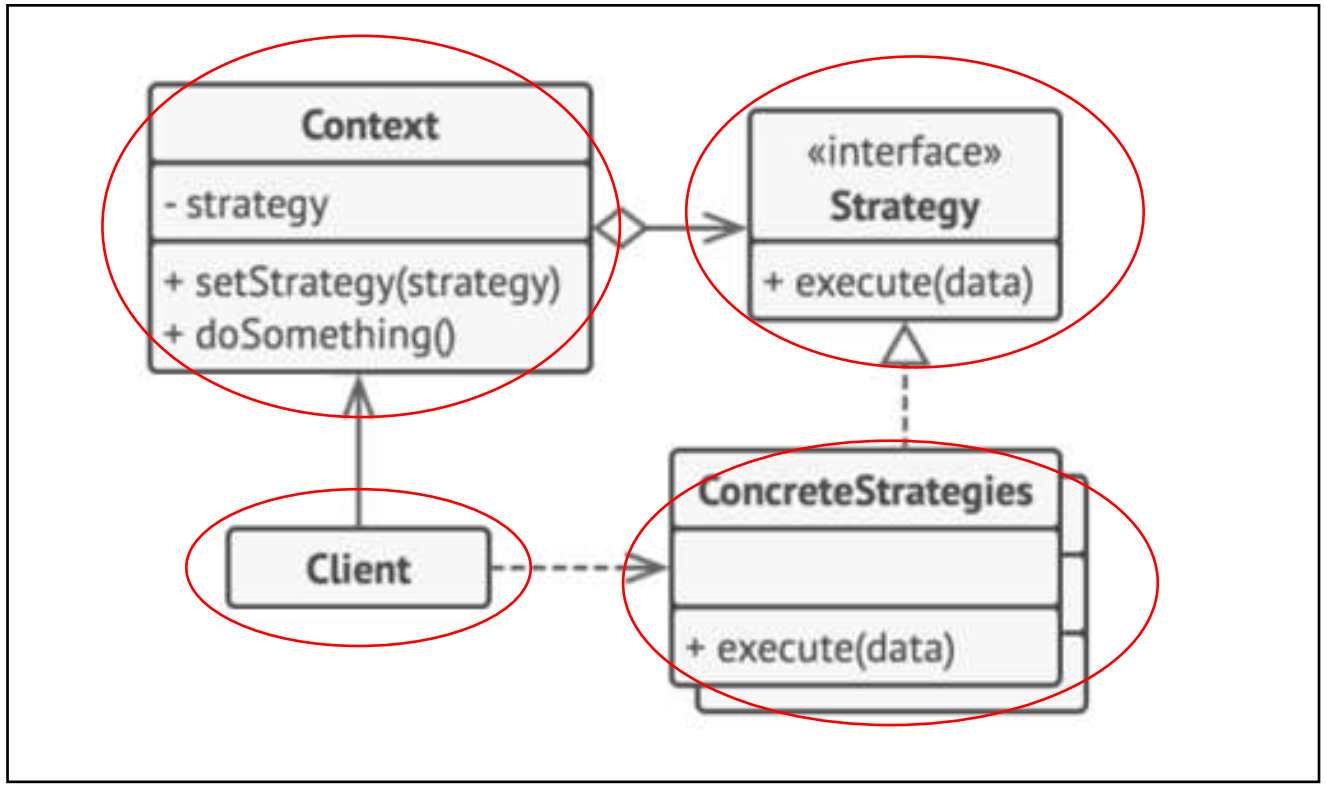
```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2
10 % 5 = 0
```

实验步骤：结合飞机大战思考策略模式结构图

请思考：

在飞机大战系统中，若采用策略模式实现不同的弹道，该如何设计？

在飞机大战中，谁是Context?



本次迭代开发目标（1）

任务清单
(CheckList)

UML类图设计

① 结合实例，按照规范绘制策略模式的结构类图

设计模式应用

② 完成代码重构，采用策略模式实现不同机型的弹道发射逻辑，以及两种火力道具的加成效果

敌机生成

③ 游戏界面可正常显示 5 种敌机，按预设周期随机出现；当玩家分数达到指定阈值时，触发 Boss 敌机生成（注意同一时间只能有1台Boss敌机）

敌机移动

④ Boss 敌机悬浮于界面上方，仅执行左右移动操作，不向下移动；

本次迭代开发目标（2）

任务清单
(CheckList)

敌机攻击

⑤ Boss 敌机采用环射弹道，单次同时发射 20 颗子弹

道具生成

⑥ Boss 敌机被击毁时，随机掉落 3 个道具（道具类型无限制）

道具效果

⑦ 火力道具生效后，英雄机将当前弹道切换为散射弹道；超级火力道具生效后，英雄机将当前弹道切换为环射弹道；炸弹、冰冻道具生效时，在控制台输出对应提示信息即可。

注：本次实验未涉及多线程相关实现，因此弹道切换后暂无法自动恢复，该功能将在实验五中进一步完善。

中期提交要求

- 提交内容

需提交项目压缩包（将整个项目打包为 zip 格式），压缩包内需包含完整**代码及 UML 类图**，确保文件完整可正常运行。

- 提交地址

提交至作业提交平台（截止日期参考平台发布），请使用统一身份认证登录，推荐使用 Chrome 浏览器。

作业平台入口： https://grader.hitsz.edu.cn/login?next=/courses

同学们，请开始实验吧！

THANK YOU